AWS File Analyzer

Multi-cloud file ingestion and AI analysis platform

Brad Asher | .NET 8 and React 19 | GitHub | Live application

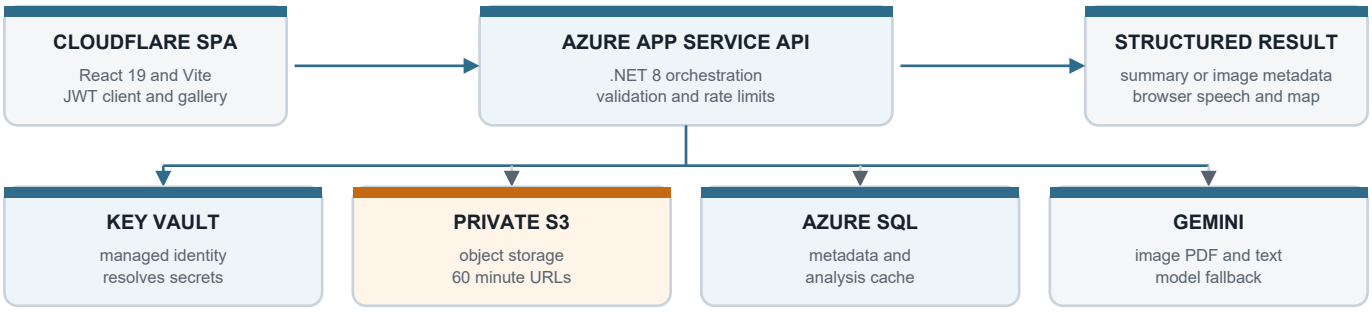
Project overview

I built a deployed document intelligence application that accepts images, PDFs, and text files, stores them privately in Amazon S3, and returns structured analysis from Google Gemini. A React single-page application runs on Cloudflare Pages, while a .NET 8 API on Azure App Service coordinates authentication, storage, model calls, and Azure SQL persistence.

The project focuses on the boundaries that make a cloud AI feature usable outside a local demo. The API validates file count, size, type, and source URL; signs temporary S3 access instead of exposing the bucket; checks a SQL cache before repeating model work; limits guest capabilities; and retries transient Gemini failures through an ordered model fallback chain.

4 providers Cloudflare Azure AWS Google	**55 of 55** local CI tests passing	**60 minutes** signed S3 URL lifetime	**8 models** cost-first Gemini fallback

Architecture



Core design

COMPONENT	IMPLEMENTATION AND PURPOSE
Ingestion and storage	The API uploads as many as five account files concurrently with Task.WhenAll. Objects remain private in S3, and 60-minute pre-signed URLs give the backend temporary read access without placing AWS credentials in the browser.
File analysis pipeline	A dispatcher selects image, PDF, or text processing. Images are sent as inline multimodal data; PdfPig extracts PDF text and summarizes long documents in 4,000-character chunks; text and HTML use a dedicated summary path.
Persistence and reuse	EF Core repositories and a Unit of Work persist upload metadata and structured analysis in Azure SQL. The service checks for an existing result before invoking Gemini and serializes database writes through a short semaphore-protected section.
Identity and operations	BCrypt credentials, Google ID tokens, JWT roles, per-route rate limits, MIME and size limits, S3 URL allowlisting, global problem responses, health checks, and Application Insights define the production boundary.

Engineering decisions and tradeoffs

I kept each provider responsible for a distinct concern and made failure handling explicit in the application layer. That produced a working cross-cloud system, while leaving several clear improvements for a larger deployment.

DECISION	WHY IT WORKS	CURRENT COST
Separate edge UI and API	Cloudflare serves the static React build, while Azure hosts the stateful .NET API, SQL access, monitoring, and managed identity.	Cross-origin configuration and two deployment pipelines must stay synchronized.
Private S3 plus signed reads	Users never receive AWS credentials and the bucket does not need public access. The server can renew links from stored object keys.	A pre-signed URL is still a bearer capability until it expires, so URL validation and short lifetimes matter.
SQL-backed result cache	Repeated analysis can return stored structured output instead of paying for another Gemini request.	The current lookup uses the temporary URL. A stable object key or content hash would improve cache identity across renewed URLs.
Parallel work with bounded writes	S3 uploads and file analyses run concurrently, while EF Core writes enter a small serialized critical section.	The in-process semaphore does not coordinate multiple API instances; database constraints and idempotent writes are the scalable boundary.
Cost-first model fallback	Transient errors, rate limits, and unavailable models trigger bounded retry and progression through eight configured Gemini Flash models.	Different fallback models can vary in latency and output. Usage telemetry is needed to tune the order with evidence.

Verification and production evidence

CHECK	LATEST RESULT	WHAT IT COVERS
Backend tests	26 of 26 passing	JWT, controllers, file validation, service dispatch, and Gemini fallback behavior
Frontend tests	29 of 29 passing	routing, auth forms, token handling, guest landing, API client, and gallery behavior
Production build	Vite build succeeds	83 modules transformed into deployable static assets
Live deployment	Frontend and API health return HTTP 200	A separate 10-scenario Playwright suite exercises guest, auth, gallery, and navigation flows

Security boundaries and next steps

Azure Key Vault supplies JWT, Gemini, and AWS secrets to App Service through system-assigned managed identity, and GitHub Actions deploys to Azure through OIDC. Guest JWTs expire after 15 minutes and can upload one file up to 2 MB; account users can upload as many as five files up to 5 MB each. The next engineering work is to add durable per-user ownership for uploads and analysis rows, replace URL-based cache identity with object keys or hashes, trace requests across all four providers with OpenTelemetry, and move from long-lived AWS keys to temporary federated credentials.

What this project demonstrates

This project required full-stack implementation across HTTP APIs, asynchronous I/O, authentication, relational persistence, cloud storage, external model reliability, infrastructure as code, CI, and public deployment. More importantly, it forced me to define trust boundaries and failure behavior between services that do not share one cloud, identity system, or availability model.